

Time Series Forecasting Project Report

Group ID: 16 Members: Simon Kistner (2668864), Jia-Sophie Sun (2383178)

04th September 2026

Abstract

We study 336-hour forecasting from 168 hours of history on a multivariate operations benchmark. We compare an adapted DLinear model with a global Temporal Convolutional Network (TCN) that incorporates known future covariates, series embeddings, and a seasonal residual forecast. On the public validation set, the TCN achieves 13.60% WAPE, outperforming DLinear at 24.05% and all evaluated baselines. We additionally evaluate both architectures on Electricity Load Diagrams through a controlled future-covariate ablation. There, strong weekly regularity makes target-history-only forecasting competitive, while engineered calendar covariates provide only modest additional gains. The code is available at https://github.com/M4GI3R/deep_learning_bonus_project_group_16.

1 Introduction

Long-horizon time-series forecasting supports planning and decision-making in domains such as energy, transportation, healthcare, and operations. Many such applications involve multiple related target series together with historical and known future covariates. Effective forecasting models must therefore capture temporal patterns, use auxiliary information, and account for systematic differences between individual series.

In this project, we forecast 96 related hourly target series. For each series, the model receives 168 hours of history and must predict the following 336 hours. The inputs additionally contain calendar variables and operational covariates that are known throughout the forecast horizon. In addition, we use the Electricity Load Diagrams [5] as an external benchmark. This setting is challenging because the prediction horizon is twice as long as the available target context, the series exhibit different operating levels, and some covariates contain missing values. Furthermore, a model trained to predict short blocks may accumulate errors when applied recursively over the complete horizon.

A central question of this work is whether a lightweight convolutional model can exploit the available covariates and temporal structure more effectively than linear and seasonal alternatives. We first establish naive and seasonal baselines and implement an adapted DLinear model [8] as a strong linear reference. Our final model is a globally trained Temporal Convolutional Network [2]. It combines causal dilated convolutions with known future covariates, learned series embeddings, missingness indicators, and a seasonal residual forecast. The complete 336-hour horizon is predicted in one forward pass without feeding predicted target values back into the model.

2 Related Work

For time-series forecasting, recurrent models are a natural starting point because they are designed to capture dependencies across sequential observations. However, recurrent-based forecasting can

face optimization difficulties, especially when learning long-range temporal dependencies, partly due to vanishing gradients during training. Variants such as Long Short-Term Memory [4] networks and Gated Recurrent Units [3] were introduced to mitigate these limitations, but the computation are still performed sequentially across time steps. This restricts parallelization during training compared with convolution-based architectures, which motivates the use of alternatives such as Temporal Convolution Networks (TCN).

TCNs [2] provide an alternative through causal and dilated convolutions with residual connections. Even though it was originally evaluated on a range of general sequence-modeling tasks, its design is also well suited to time-series forecasting, as the causal convolutions prevent access to future values while dilation allow the receptive field to grow efficiently and thereby capture long-range temporal dependencies. These properties have subsequently been incorporated into models for multivariate forecasting settings. For example, M-TCN [6] extends TCN architecture with an additional multi-head architecture, while another approach combines TCN-based probabilistic forecasting with Bayesian optimization [1].

At the same time, research in time-series forecasting has shown that even simpler linear and MLP-based architecture can perform competitively. DLinear [8], for example, decomposes a time series into trend and remainder components and applies a single linear layer per component. Despite its simplicity, it outperformed transformer-based models such as Informer [9] and Autoformer [7] on several long-term forecasting benchmarks. However, its linear nature limits its ability to represent nonlinear temporal relationships and its commonly used channel-independent formulation does not explicitly model dependencies between variables [8].

These trade-offs motivate our use of DLinear as a lightweight linear reference and a TCN-based architecture as the more expressive and therefore our main model.

3 Method

Both models, an adapted DLinear and a TCN-based model, are trained globally: their respective parameters are shared across all 96 target series, while a learned series embedding allows them to represent unit-specific behavior. Given 168 hours of context, both models predict the complete 336-hour horizon in a single forward pass.

3.1 Forecasting Setup and Input Representation

The historical and known covariates comprise calendar information, trend and workload variables, operational forecasts, maintenance indicators, capacity information, and series-level spatial features.

Target values are standardized per series using the mean and standard deviation of the corresponding training observations. This prevents high-volume series from dominating the shared model while retaining their original scale for metric computation. Covariates are standardized globally using statistics calculated exclusively on the training split.

Some released covariates contain missing values. After standardization, these values are replaced by zero, which corresponds to mean imputation in the standardized space. For every covariate containing missing training values, we add a binary missingness indicator. This allows the model to distinguish an imputed value from a genuinely average observation.

Each series is additionally represented by a learned 16-dimensional embedding. The shared model can therefore learn common temporal patterns across all series while retaining information about systematic differences between individual units.

3.2 DLinear-Based Model

DLinear [8] serves as our lightweight linear reference. Following the original architecture, the input target sequence is decomposed into a smooth trend component and a remainder component using a moving average. Separate linear layers then project both components directly from the 168-hour context to the 336-hour forecast horizon.

Let $x \in R^{168}$ denote a standardized target history and let $\ell = x_{168}$ be its most recent value. We first center the sequence as $z = x - \ell$ and compute

$$z_{\text{trend}} = \text{AvgPool}_{25}(z), \quad z_{\text{seasonal}} = z - z_{\text{trend}}. \quad (1)$$

The target-only forecast is then

$$\hat{y}_{\text{linear}} = W_{\text{seasonal}} z_{\text{seasonal}} + W_{\text{trend}} z_{\text{trend}} + \ell, \quad (2)$$

where both linear transformations map from 168 to 336 time steps.

The decomposition and the two temporal projections follow DLinear. We introduce two task-specific adaptations. First, last-value centering follows the motivation of NLinear [8] and makes the forecast less sensitive to a shift in the target level between the training data and forecast origin. Second, the original target-only forecast is extended with a small nonlinear correction network for the known future covariates:

$$\hat{y} = \hat{y}_{\text{linear}} + g(c_{\text{future}}, e_s), \quad (3)$$

where c_{future} denotes the known future covariates, e_s is the series embedding, and g is a shared two-layer network that produces a correction for each forecast step. It contains a 64-unit hidden layer, GELU activation, dropout, and a scalar output. Its final layer is initialized to zero, so training starts from the target-only DLinear forecast and gradually learns covariate- and series-dependent corrections.

3.3 Temporal Convolutional Network

Our final model is based on the generic TCN architecture proposed by Bai et al. [2]. We retain its principal components: causal convolutions, exponentially increasing dilation, two convolutions per residual block, weight normalization, ReLU activations, spatial dropout, and residual connections.

The model contains seven residual blocks with 64 channels and kernel size three. Block i uses dilation 2^i , resulting in dilations 1, 2, 4, 8, 16, 32, and 64. Causal cropping after each convolution prevents the representation at any position from accessing future inputs. With two convolutions per residual block, the receptive field is

$$R = 1 + 2(k-1) \sum_{i=0}^6 2^i = 509. \quad (4)$$

It therefore covers the complete sequence of 168 historical and 336 future positions. We use spatial dropout with probability 0.15. The residual-block structure is shown in Figure A.1b.

Since the original TCN was introduced as a generic sequence model, we need to adapt its input and output representation to incorporate known future covariates. For the historical portion, the target channel contains the observed standardized target values. For the future portion, we repeat the last 24 observed target values across the forecast horizon, producing a simple daily seasonal forecast. Both covariates are concatenated with this target channel, and an additional binary feature indicates whether a position belongs to the historical context or forecast horizon.

A 1×1 convolution projects these inputs to 64 channels before they enter the residual TCN blocks. The hidden representations corresponding to the 336 future positions are passed to the prediction head together with the series embedding. The head, shown in A.1c, contains a 64-unit hidden layer, ReLU activation, dropout, and a scalar output. Rather than predicting the complete target independently, the network learns a correction to the seasonal forecast:

$$\hat{y}_{t+h} = y_{t+h-24}^{\text{seasonal}} + \Delta_{t+h}, \quad h = 1, \dots, 336. \quad (5)$$

The final output layer is initialized to zero, so the initial model reproduces the seasonal forecast and subsequently learns corrections based on the target history, future covariates, and series identity.

The complete architecture diagram is provided in A.1.

3.4 Training Objective

The forecasting task is evaluated using absolute, squared, and relative error metrics. We therefore optimize a composite loss combining scale-normalized versions of MAE, MSE, RMSE, MAPE, sMAPE, and WAPE:

$$\mathcal{L} = \frac{1}{6} \left(\frac{\text{MAE}}{E[|y|] + \varepsilon} + \frac{\text{MSE}}{E[y^2] + \varepsilon} + \frac{\text{RMSE}}{\sqrt{E[y^2]} + \varepsilon} + \text{MAPE} + \text{sMAPE} + \text{WAPE} \right), \quad (6)$$

where E denotes the empirical average over all target values in the current training batch and $\varepsilon = 10^{-6}$ is used in metric denominators for numerical stability.

The absolute and squared components encourage accurate predictions while penalizing large errors, whereas the percentage-based components balance performance across target values of different magnitudes. Normalization keeps the individual terms on comparable scales. Although the models operate on standardized targets, the loss is calculated after transforming predictions and targets back to their original scale.

3.5 Training Procedure

Training examples are generated with a sliding window consisting of 168 context steps and 336 prediction steps. Consecutive windows are separated by a stride of 12 hours for the operations benchmark and 96 hours for the substantially larger Electricity Load Diagrams dataset to reduce redundant overlap and computation. The stride remains fixed across all variants within each dataset. Both models are trained using AdamW with weight decay 10^{-4} , gradient clipping at norm 1.0, and random seed 16.

DLinear uses a learning rate of 0.001 and batch size 256. The TCN uses a learning rate of 0.0005 and batch size 128. Training is limited to 200 epochs and stopped when the local validation objective does not improve for 20 epochs. The local validation set consists of the final 336 hours of every series, matching the complete benchmark forecast horizon.

The best local checkpoint was obtained after 22 epochs for DLinear and 17 epochs for the TCN. For the public forecast, each model was trained again from random initialization on all available training observations for its selected number of epochs. Since the target values are non-negative, final predictions are clipped at zero after transformation back to the original scale.

3.6 Relation to the Exposé

The final approach follows the main direction proposed in the exposé: DLinear is used as a lightweight neural reference and a TCN with known future covariates and learned series embeddings

serves as the principal architecture. The main decision left open in the exposé concerned iterative versus full-horizon prediction. Based on the behavior of the initial iterative prototypes, we selected a single-shot 336-step output for both final models. A TSMixer-style architecture was considered as a fallback but was not pursued after the adapted TCN achieved strong local and public validation performance.

4 Experiments

We evaluate DLinear and the TCN on the provided operations benchmark and on Electricity Load Diagrams as an additional dataset.

4.1 Provided Operations Benchmark

For the provided benchmark, we report the following validation result:

Method	WAPE(%)	Notes
Naive last-value baseline	48.10%	minimum baseline
Seasonal mean baseline	34.41%	stronger reference
DLinear	24.05%	lightweight linear reference
TCN-based	13.60%	final model

Table 1: Summary of public validation results on the operations benchmark. Lower WAPE is better. (Complete results across all evaluation metrics are reported in Appendix Table B.1)

Both learned models substantially outperform the supplied baselines. DLinear reduces WAPE by approximately 50% relative to the naive last-value baseline. The TCN performs best for every reported metric, reducing WAPE by approximately 72% relative to the naive baseline and 60% relative to the seasonal-mean baseline. This supports the hypothesis that the TCN uses the supplied future operational covariates more effectively than the linear reference.

4.2 Additional Dataset: Electricity Load Diagrams

The dataset *Electricity Load Diagrams* [5] (ELD) contains electricity-consumption series from Portuguese clients between 2011 and 2014. Following the Hugging Face `lstnet` configuration, we retain 320 series from 2012–2014 and sum the original 15-minute observations onto an hourly grid. We forecast these hourly-resampled load values using the same 168-hour context and 336-hour forecast horizon as for the operations benchmark.

Unlike the provided operations dataset, ELD supplies no dynamic future covariates: it contains the target histories, start timestamps, series identifiers, and a static series category. We therefore evaluate three feature variants while keeping the target representation, forecasting models, and evaluation horizon unchanged:

- **E1 Raw:** no engineered future covariates.
- **E2 Matched:** the deterministic calendar features that match those supplied with the operations dataset: `hour_sin`, `hour_cos`, `dow_sin`, `dow_cos`, `is_weekend`, and a linear `trend`.
- **E3 Extended:** all E2 features plus `day_of_year_sin`, `day_of_year_cos`, `month_sin`, `month_cos`, a Portuguese public-holiday indicator, and a daylight-saving-time transition indicator.

All engineered variables are deterministic from the timestamp and are therefore known throughout the forecast horizon. The three variants form a controlled feature ablation: E1 measures target-history-only performance, E2 tests whether the calendar representation transfers from the operations benchmark, and E3 tests whether additional annual and Portugal-specific calendar information is beneficial.

Method	WAPE	Notes
Naive last value	36.31%	minimum baseline
Seasonal mean	27.92%	seasonal reference
Lag-24 repeat	12.09%	daily repetition
Lag-168 repeat	11.05%	best baseline
DLinear, E1 Raw	11.38%	no future covariates
TCN, E1 Raw	11.11%	no future covariates
DLinear, E2 Matched	11.33%	matched calendar features
TCN, E2 Matched	10.82%	matched calendar features
DLinear, E3 Extended	11.03%	extended calendar features
TCN, E3 Extended	10.03%	extended calendar features

Table 2: WAPE summary on the ELD validation horizon. Lower is better.
(Complete results across all evaluation metrics are reported in Appendix Table B.2)

Compared with the operations benchmark, the ELD results show much smaller differences between learned models and simple baselines. The naive and seasonal-mean baselines perform poorly, whereas repeating the previous day or week is already highly competitive. In particular, the strong Lag-168 result suggests that the series are dominated by stable weekly patterns and are largely predictable from their target histories alone.

Accordingly, both DLinear and the TCN achieve competitive results even in the raw experiment, but neither clearly outperforms Lag-168 across all metrics. The matched calendar features yield only minimal changes: WAPE improves by less than 0.3% for either model, with similarly modest changes in the other relative metrics. Such small differences could reflect ordinary training variation and should not be interpreted as strong evidence that the matched features improve generalization.

The extended feature set achieves the best WAPE, improving over the raw variant by 0.35% for DLinear and 1.08% for the TCN. These limited gains are consistent with the strong Lag-168 baseline: much of the predictable structure is already present in the target history, leaving less room for the engineered calendar covariates. However, the effect is not consistent across all metrics (Appendix Table B.2). The ablation therefore provides only tentative evidence of a benefit, which should be confirmed through repeated runs or additional validation periods.

5 Conclusion

We developed two direct multi-horizon forecasting models: an adapted DLinear reference and a covariate-aware TCN. On the operations benchmark, both models outperform the supplied baselines, with the TCN achieving the strongest public validation results. On ELD, however, both raw models are already competitive with a strong weekly baseline. Engineered calendar covariates improve the TCN more clearly than DLinear, but the gains remain limited and are not uniform across all metrics. These results suggest that the value of future covariates depends strongly on the temporal structure of the dataset. Since the ablation uses one seed and one validation horizon, repeated runs and additional evaluation periods are needed to confirm the observed feature effects.

Contributions

All team members contributed equally to the project. The implementation was developed collaboratively and the experiments, analysis, and report were completed through an equally shared effort.

References

- [1] Probabilistic forecasting with temporal convolutional neural network. *Neurocomputing*, 399:491–501, 2020.
- [2] Shaojie Bai, J. Zico Kolter, and Vladlen Koltun. An empirical evaluation of generic convolutional and recurrent networks for sequence modeling. *arXiv preprint arXiv:1803.01271*, 2018.
- [3] Kyunghyun Cho, Bart Van Merriënboer, Çağlar Gulçehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using rnn encoder–decoder for statistical machine translation. In *Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP)*, pages 1724–1734, 2014.
- [4] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural computation*, 9(8):1735–1780, 1997.
- [5] Artur Trindade. Electricityloadaddiagrams20112014, 2015.
- [6] Renzhuo Wan, Shuping Mei, Jun Wang, Min Liu, and Fan Yang. Multivariate temporal convolutional network: A deep neural networks approach for multivariate time series forecasting. *Electronics*, 8(8):876, 2019.
- [7] Haixu Wu, Jiehui Xu, Jianmin Wang, and Mingsheng Long. Autoformer: Decomposition transformers with auto-correlation for long-term series forecasting. *Advances in neural information processing systems*, 34:22419–22430, 2021.
- [8] Ailing Zeng, Muxi Chen, Lei Zhang, and Qiang Xu. Are transformers effective for time series forecasting? *Proceedings of the AAAI Conference on Artificial Intelligence*, 37(9):11121–11128, 2023.
- [9] Haoyi Zhou, Shanghang Zhang, Jieqi Peng, Shuai Zhang, Jianxin Li, Hui Xiong, and Wancai Zhang. Informer: Beyond efficient transformer for long sequence time-series forecasting. In *Proceedings of the AAAI conference on artificial intelligence*, volume 35, pages 11106–11115, 2021.

Appendix

A TCN Architecture

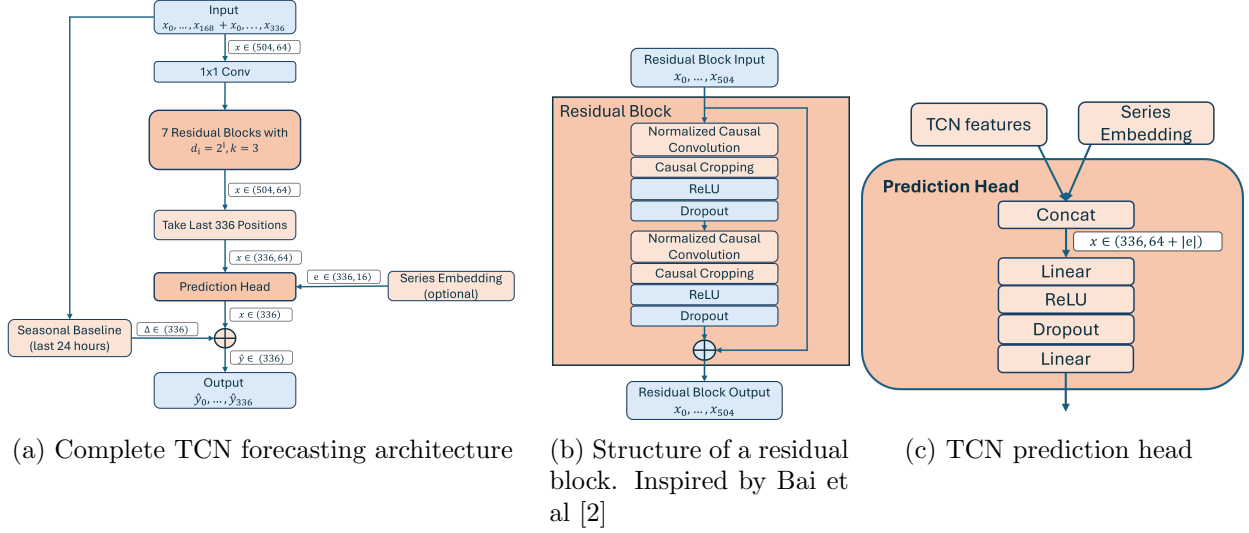


Figure A.1: Architecture of the proposed TCN forecasting model. The complete architecture is shown in (a), while (b) details the causal convolutional residual blocks and (c) shows the prediction head that combines the TCN features with the learned series embedding.

B Supplementary Results and Model Architecture

B.1 Complete Operations Benchmark Results

Method	MAE	MSE	RMSE	MAPE (%)	sMAPE (%)	WAPE (%)
Naive last value	5.29	48.61	6.97	61.35	52.71	48.10
Seasonal mean	3.79	27.10	5.21	45.68	37.43	34.41
Lag-24 repeat	5.15	43.59	6.60	78.51	47.31	46.82
DLinear	2.65	13.84	3.72	35.57	26.31	24.05
TCN-based	1.50	6.39	2.53	16.68	15.19	13.60

Table B.1: Complete public validation results on the operations benchmark. Lower is better for every metric.

B.2 Complete Electricity Load Diagrams Results

Method	MAE	MSE	RMSE	MAPE (%)	sMAPE (%)	WAPE (%)
Naive last value	761.86	15,035,126.92	3,877.52	49.38	41.11	36.31
Seasonal mean	585.88	29,038,365.78	5,388.73	36.23	26.29	27.92
Lag-24 repeat	253.77	2,178,487.20	1,475.97	30.37	18.31	12.09
Lag-168 repeat	231.79	2,278,374.78	1,509.43	22.62	15.37	11.05
DLinear, E1 Raw	238.84	2,185,789.86	1,478.44	26.71	18.38	11.38
TCN, E1 Raw	233.05	2,047,219.69	1,430.81	23.26	17.33	11.11
DLinear, E2 Matched	237.79	2,007,308.23	1,416.80	26.15	18.56	11.33
TCN, E2 Matched	227.07	1,888,168.86	1,374.11	22.97	16.54	10.82
DLinear, E3 Extended	231.49	1,811,312.08	1,345.85	24.93	18.28	11.03
TCN, E3 Extended	210.36	1,379,460.09	1,174.50	21.40	17.63	10.03

Table B.2: Complete validation results for Electricity Load Diagrams. Absolute metrics are reported in the dataset’s native target scale. Lower is better for every metric.